Day 123 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 18, Day 5 reference

Watch: StatQuest: Principal Component Analysis (PCA), Step-by-Step (StatQuest with Josh Starmer) | PCA and SVD - the math explained simply (TileStats)

`day123_pca.py`, segment by segment, in the order `main()` uses things. The fit, projection, reconstruction, power iteration and variance helpers are new; Day 115's kNN and Day 119's k-means are imported unchanged. scikit-learn's PCA appears only as an outside check. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: mu = the mean of each feature · C = the covariance matrix · V = the components as columns · vals = the eigenvalues, largest first · Z = coordinates along the components

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE); sys.path.insert(0, os.path.join(HERE, "..",
    "week-17-classical-ml-kernels-ensembles"))
with contextlib.redirect_stdout(io.StringIO()):
    import day119_k_means as d119
    import day115_k_nearest_neighbors as d115
ari, standardise = d119.ari, d119.standardise
```

- `sys.path.insert(0, os.path.join(HERE, "..", "week-17-classical-ml-kernels-ensembles"))` — Day 115 lives in last week's folder.
- `import day115_k_nearest_neighbors as d115` — its kNN is used unchanged on the components.

Segment 1 — PCA

pca_fit, transform and reconstruct

MATH

$$C = A^T A/(n-1); \quad Z = (X-\mu)V_k; \quad X_{rebuilt} = ZV_k^T + \mu$$

CODE

```
def pca_fit(X):
    mu = X.mean(0); A = X - mu
    C = A.T @ A / (len(X) - 1)
    vals, vecs = np.linalg.eigh(C)
    order = np.argsort(vals)[::-1]
    return mu, np.maximum(vals[order], 0), vecs[:, order]
def transform(X, mu, V, k):
    return (X - mu) @ V[:, :k]
def reconstruct(Z, mu, V, k):
    return Z @ V[:, :k].T + mu
```

- `A.T @ A / (len(X) - 1)` — the covariance of every pair of features at once.
- `np.linalg.eigh(C)` — eigh is for symmetric matrices: real eigenvalues, perpendicular eigenvectors, in ascending order.

- `np.argsort(vals)[::-1]` — the positions of the eigenvalues, largest first; used to reorder both outputs.
- `V[:, :k]` — the first k columns: the first k components.

power_iteration

MATH

$$v_{t+1} = Cv_t/|Cv_t|; \quad \lambda = v^T C v; \quad C \leftarrow C - \lambda v v^T$$

CODE

```
def power_iteration(C, k, iters=5000, tol=1e-12, seed=0):
    rng = np.random.RandomState(seed); C = C.copy(); vals, vecs, steps = [], [], []
    for _ in range(k):
        v = rng.normal(size=len(C)); v /= np.linalg.norm(v)
        for t in range(1, iters + 1):
            w = C @ v; w /= np.linalg.norm(w)
            done = np.linalg.norm(w - v) < tol
            v = w
            if done:
                break
        lam = float(v @ C @ v); vals.append(lam); vecs.append(v); steps.append(t)
        C -= lam * np.outer(v, v)
    return np.array(vals), np.array(vecs).T, steps
```

- `w = C @ v; w /= np.linalg.norm(w)` — one step: multiply, then rescale to length 1.
- `done = ...; v = w; if done: break` — the test is made before v is replaced, so the loop stops on the step that converged.
- `C -= lam * np.outer(v, v)` — deflation: subtracts the found direction, so the next run finds the next one.
- `np.array(vecs).T` — a list of vectors becomes rows; .T makes them columns, like eigh's output.

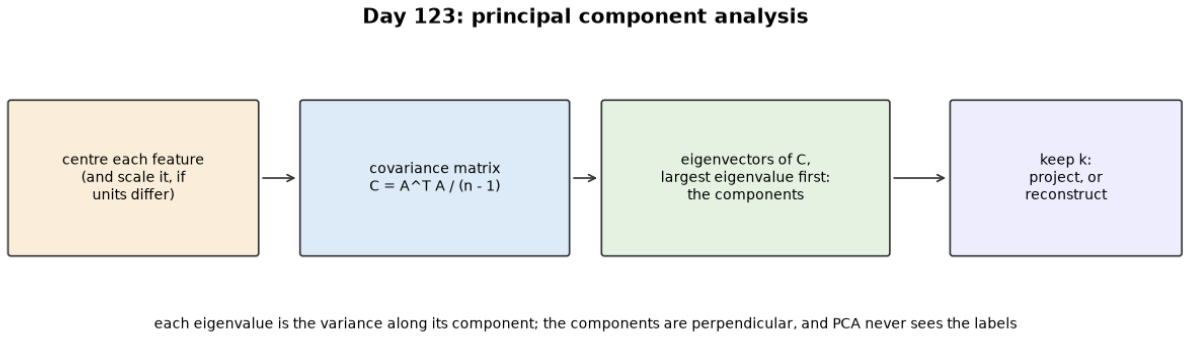
explained and needed

CODE

```
def explained(vals):
    return vals / vals.sum()
def needed(vals, share):
    return int(np.searchsorted(np.cumsum(explained(vals)), share - 1e-12) + 1)
```

- `np.cumsum(...)` — running totals: the share kept by the first 1, 2, 3, ... components.
- `np.searchsorted(cum, share)` — the first position where the running total reaches the share; + 1 turns a position into a count.

SYSTEM DIAGRAM



The PCA pipeline.

Segment 2 — The experiments

demo_check: comparing directions that may differ in sign

CODE

```
X, _ = sets["digits"]; mu, vals, V = pca_fit(X); sk = PCA().fit(X)
print(f"digits: explained-variance shares, largest gap from scikit-learn {np.abs(explained(vals) -
    sk.explained_variance_ratio_).max():.1e}; "
      f"first 10 components, smallest |cos| with scikit-learn's {np.abs((V[:, :10] *
          sk.components_[:10].T).sum(0)).min():.10f} (signs may differ)")
```

- `sk.components_[:10].T` — scikit-learn stores components as rows; .T makes them columns like ours.
- `np.abs((A * B).sum(0))` — column-by-column dot products, made positive: a component and its negative are the same direction.

demo_misleads: data made so the answer is known

CODE

```
rng = np.random.RandomState(0); y = np.r_[np.zeros(200, int), np.ones(200, int)]
X = np.c_[rng.normal(0, 3.0, 400), np.where(y == 1, 0.5, -0.5) + rng.normal(0, 0.2, 400)]
```

- `np.where(y == 1, 0.5, -0.5)` — +0.5 for one class and -0.5 for the other: the whole signal.
- `rng.normal(0, 3.0, 400)` — a wide column of pure noise, which PCA will rank first.

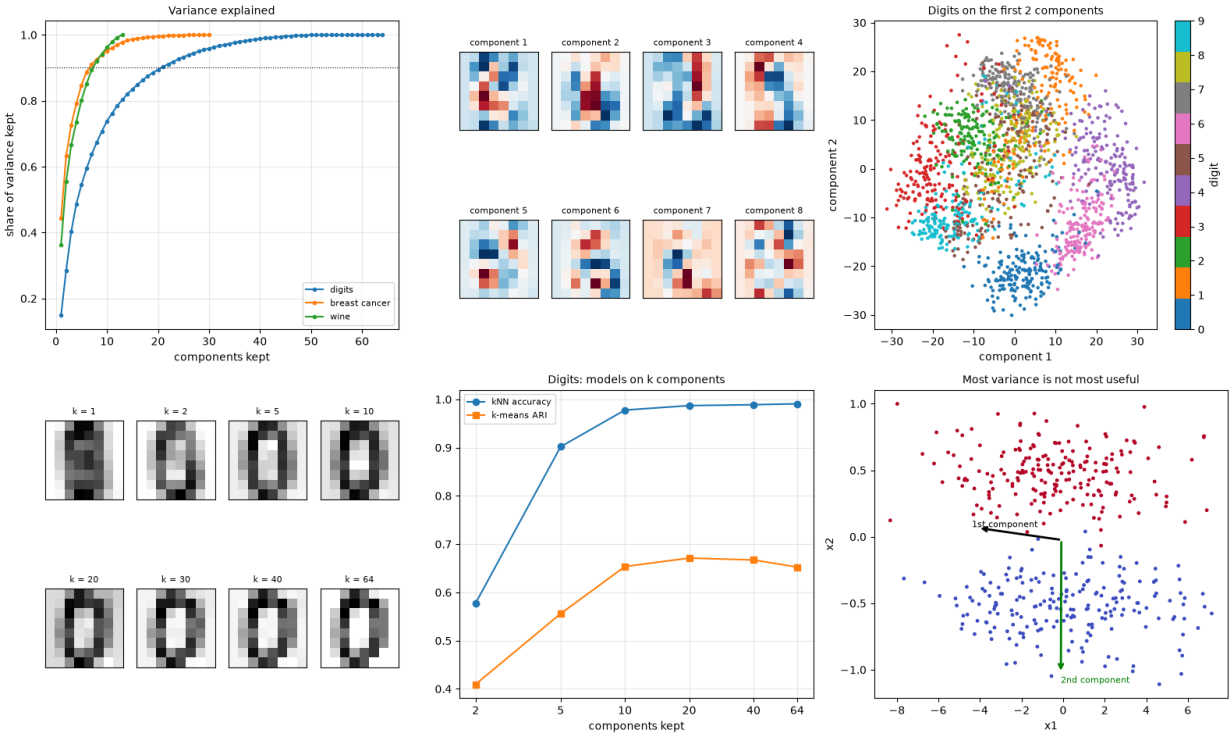
plot_results: images from vectors, and a log axis without clutter

CODE

```
sub = G[0, 1].subgridspec(2, 4)
for j in range(8):
    b = fig.add_subplot(sub[j // 4, j % 4]); b.imshow(V[:, j].reshape(8, 8), cmap="RdBu_r");
        b.set_xticks([]); b.set_yticks([])
    b.set_title(f"component {j + 1}", fontsize=8)
```

- `V[:, j].reshape(8, 8)` — a component is 64 weights, one per pixel: reshaped, it is an image.
- `cmap="RdBu_r"` — a diverging colour map: red and blue for positive and negative weights.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.linalg.eigh(C)` — eigenpairs of a symmetric matrix, ascending.
- `np.argsort(v)[::-1]` — positions, largest first.
- `np.outer(v, v)` — the matrix $v\,v^T$.
- `np.cumsum(a)` — running totals.
- `np.searchsorted(a, x)` — where x would go in a sorted array.
- `ax.minorticks_off()` — drop a log axis's minor labels.

End of Day 123 syntax reference. For the experiments and what they showed, see the Day 123 complete guide.